

UNIVERSIDAD DEL VALLE DE GUATEMALA

PROGRAMACIÓN ORIENTADA A OBJETOS

DIEGO ISAIAS MARROQUÍN MOSCOSO

LABORATORIO #1

Requisitos funcionales del sistema:

NOTA: Algunas implementaciones fueron hechas por criterio propio como la penalización del 10% si no se activa la digievolucion, ya que el documento no explicaba la diferencia entre ataque normal o digievolucion si pasaba algo, lo mismo con la parte de la defensa, era muy ambigua y no de que forma se usaba y así con mas cosas.

- El sistema debe crear ocho Digimon disponibles, cada uno con nombre, tipo, ataque, defensa y una Digievolución asociada.
- El sistema debe manejar los tipos Fuego, Agua, Planta y Eléctrico, aplicando ventajas, desventajas o neutralidad durante el cálculo del ataque total.
- El sistema debe permitir ingresar el nombre de dos entrenadores.
- Cada entrenador debe seleccionar cuatro Digimon diferentes y un Digimon seleccionado por un entrenador no puede ser escogido por el otro.
- El sistema debe permitir jugar una batalla compuesta por cuatro rondas.

- En cada ronda cada entrenador debe elegir uno de sus Digimon que todavía no haya sido utilizado en esa batalla.
- Cada entrenador debe decidir en cada ronda si desea intentar utilizar la Digievolución de su Digimon.
- La Digievolución debe activarse de acuerdo con su porcentaje de probabilidad y puede aumentar ataque, incrementar defensa o reducir el ataque del rival.
- Si una Digievolución no se activa, el Digimon recibe una penalización del 10% de su ataque para esa ronda.
- Cuando una Digievolución se activa, su efecto se aplica en la ronda actual y se conserva durante una ronda adicional.
- La defensa reduce el ataque rival utilizando la regla implementada de un punto de reducción por cada tres puntos de defensa.
- El sistema debe calcular el ataque total de ambos Digimon y determinar el ganador de cada ronda; si los ataques totales son iguales, la ronda termina en empate.
- El sistema debe acumular las rondas ganadas y determinar al ganador final después de las cuatro rondas.
- El sistema debe permitir reiniciar los datos temporales de batalla para jugar nuevamente con los mismos equipos, conservando las victorias acumuladas de los entrenadores.
- El menú principal debe permitir iniciar una batalla, consultar la información de los entrenadores, consultar la información de los Digimon y terminar el programa.

Digievolucion
- nombre: String - efecto: String - descripcion: String - valorEfecto: int - probabilidad: int
+ Digievolucion(nombre: String, efecto: String, descripcion: String, valorEfecto: int, probabilidad: int) + getNombre(): String + getEfecto(): String + getValorEfecto(): int + getprobabilidad(): int + intentarActivar(): boolean + toString(): String

Propósito de la clase: Esta clase representa la Digievolución que puede utilizar un Digimon durante una ronda. Su función es almacenar el nombre de la evolución, el tipo de efecto que produce, una descripción, el valor numérico del efecto y su probabilidad de activación. También contiene la lógica aleatoria que determina si la Digievolución logra activarse.

Propósito de los atributos:

- nombre: almacena el nombre de la Digievolución, por ejemplo Adult, Mega o Ultimate. Permite identificar qué evolución posee cada Digimon.
- efecto: almacena el tipo de modificación que produce la Digievolución. En el programa puede ser Ataque, Incrementar Defensa o Reduccion y es utilizado por BatallaDigimon para saber qué cálculo realizar.
- descripcion: almacena una explicación textual del efecto de la Digievolución. Es útil para mostrar al usuario qué hace la habilidad.

- valorEfecto: almacena la cantidad numérica que se suma o resta cuando la Digievolución se activa, por ejemplo 15 puntos de ataque o 20 puntos de defensa.

- probabilidad: almacena el porcentaje de posibilidad de que la Digievolución se active. Es utilizado por intentarActivar() para decidir el resultado de forma aleatoria.

Propósito de los métodos:

- Constructor Digievolucion(nombre, efecto, descripcion, valorEfecto, probabilidad): recibe todos los datos necesarios de una Digievolución y los asigna a sus atributos para dejar el objeto listo para utilizarse.

- getNombre(): no requiere parámetros. Retorna el nombre de la Digievolución para identificarla o mostrarla en pantalla.

- getEfecto(): no requiere parámetros. Retorna el tipo de efecto para que BatallaDigimon determine si debe modificar ataque, defensa o el ataque del rival.

- getValorEfecto(): no requiere parámetros. Retorna el valor numérico del efecto para aplicarlo durante la batalla.

- getprobabilidad(): no requiere parámetros. Retorna el porcentaje de activación almacenado.

- intentarActivar(): no requiere parámetros. Genera un número aleatorio entre 1 y 100, lo compara con probabilidad y retorna verdadero si la Digievolución se activa o falso si falla.

- **toString():** no requiere parámetros. Retorna en forma de texto el nombre, efecto, probabilidad y descripción de la Digievolución para mostrar su información.

Digimon
- nombre: String - tipo: String - ataque: int - defensa: int - seleccionado: boolean - usado: boolean - digievolucion: Digievolucion
+ Digimon(nombre: String, tipo: String, ataque: int, defensa: int, digievolucion: Digievolucion) + getNombre(): String + getTipo(): String + getAtaque(): int + getDefensa(): int + getDigievolucion(): Digievolucion + getUsado(): boolean + getSeleccionado(): boolean + setUsado(valor: boolean): void + setSeleccionado(valor: boolean): void + toString(): String

Propósito de la clase: Esta clase representa a cada criatura que puede ser seleccionada y utilizada en las batallas. Almacena sus estadísticas principales, su tipo, su estado de selección y uso, y la Digievolución que tiene asociada. Es el objeto principal que los entrenadores escogen para formar sus equipos y combatir.

Propósito de los atributos:

- nombre: almacena el nombre identificador del Digimon y permite mostrar qué criatura fue seleccionada o está combatiendo.
- tipo: almacena el elemento del Digimon, como Fuego, Agua, Planta o Electrico. Es necesario para calcular ventajas y desventajas de tipo.
- ataque: almacena el valor base ofensivo del Digimon y sirve como punto de partida para calcular el ataque total de cada ronda.
- defensa: almacena la capacidad defensiva del Digimon. En la batalla se divide entre tres para determinar cuánto reduce el ataque del rival.
- seleccionado: indica si el Digimon ya fue escogido por alguno de los entrenadores al formar los equipos. Evita que ambos entrenadores seleccionen el mismo Digimon.
- usado: indica si el Digimon ya participó en una ronda de la batalla actual. Evita que un entrenador repita el mismo Digimon durante las cuatro rondas.
- digievolucion: almacena una referencia al objeto Digievolucion asociado al Digimon, permitiendo consultar e intentar activar su habilidad especial.

Propósito de los métodos:

- Constructor Digimon(nombre, tipo, ataque, defensa, digievolucion): recibe los datos principales del Digimon y su Digievolución. Además inicializa seleccionado y usado en falso porque el Digimon todavía no ha sido escogido ni utilizado.

- getNombre(): retorna el nombre del Digimon.
- getTipo(): retorna el tipo del Digimon para realizar las comparaciones de tipo.
- getAtaque(): retorna el ataque base utilizado en el cálculo de la ronda.
- getDefensa(): retorna la defensa base utilizada para reducir el ataque rival.
- getDigievolucion(): retorna el objeto Digievolucion asociado para consultar o activar su efecto.
- getUsado(): retorna si el Digimon ya participó en una ronda.
- getSeleccionado(): retorna si el Digimon ya pertenece al equipo de algún entrenador.
- setUsado(valor): recibe un booleano y actualiza el estado usado. Se utiliza al seleccionar un Digimon para una ronda y al reiniciar una batalla.
- setSeleccionado(valor): recibe un booleano y actualiza el estado seleccionado. Se utiliza cuando un entrenador escoge el Digimon para su equipo.
- toString(): retorna una representación textual con nombre, ataque, defensa, tipo, disponibilidad y la información de su Digievolución.

Entrenador
- nombre: String - rondasGanadas: int - digimones: Digimon[] - victorias: int
+ Entrenador(nombre: String, digimones: Digimon[]) + getNombre(): String + getRondasGanadas(): int + getDigimones(): Digimon[] + ResetearRondas(): void + resetearDigimones(): void + incrementarRonda(): void + incrementarVictoria(): void + elegirDigimon(posicion: int): Digimon + toString(): String

Propósito de la clase: Esta clase representa a cada jugador o entrenador que participa en el sistema. Mantiene su nombre, el equipo de cuatro Digimon, las rondas ganadas durante la batalla actual y las victorias acumuladas en partidas completas. También controla que sus Digimon no se repitan dentro de una batalla.

Propósito de los atributos:

- nombre: almacena el nombre ingresado por el jugador y permite identificar al entrenador en menús, rondas y resultados.
- rondasGanadas: almacena cuántas rondas ha ganado el entrenador dentro de la batalla actual. Se utiliza para determinar al ganador después de cuatro rondas.
- digimones: almacena el arreglo de cuatro Digimon que fueron seleccionados para formar el equipo del entrenador.
- victorias: almacena cuántas batallas completas ha ganado el entrenador. Este valor se conserva aunque se reinicie una batalla.

Propósito de los métodos:

- Constructor Entrenador(nombre, digimones): recibe el nombre y el arreglo de Digimon del equipo. Inicializa rondasGanadas y victorias en cero.
- getNombre(): retorna el nombre del entrenador.
- getRondasGanadas(): retorna la cantidad de rondas ganadas en la batalla actual para que BatallaDigimon pueda determinar al ganador final.
- getDigimones(): retorna el arreglo de Digimon que pertenece al entrenador.
- ResetearRondas(): no requiere parámetros. Coloca rondasGanadas nuevamente en cero para comenzar una nueva batalla.
- resetearDigimones(): recorre los Digimon del equipo y coloca su atributo usado en falso para que puedan volver a participar en otra batalla.
- incrementarRonda(): aumenta en uno las rondas ganadas cuando el entrenador vence una ronda.
- incrementarVictoria(): aumenta en uno las victorias del entrenador cuando gana una batalla completa.
- elegirDigimon(posicion): recibe la posición del Digimon dentro del equipo. Si no ha sido usado, lo marca como usado y lo retorna; si ya fue utilizado retorna null.

- *toString()*: retorna un texto con el nombre del entrenador, sus victorias y los nombres de los Digimon que forman su equipo.

BatallaDigimon
- digimonesDisponibles: Digimon[] - jugador1: Entrenador - jugador2: Entrenador - rondaActual: int - ataqueFinaU1: int - ataqueFinaU2: int - digievolucionActivadaJ1: boolean - digievolucionActivadaJ2: boolean - efectoAnteriorDEvolucionJ1: int - efectoAnteriorDEvolucionJ2: int - tipoEfectoAnteriorDEvolucionJ1: String - tipoEfectoAnteriorDEvolucionJ2: String
+ BatallaDigimon(digimonesDisponibles: Digimon[], jugador1: Entrenador, jugador2: Entrenador) + getAtaqueFinaU1(): int + getAtaqueFinaU2(): int + getDigievolucionActivadaJ1(): boolean + getDigievolucionActivadaJ2(): boolean + getDigimonesDisponibles(): Digimon[] + getJugador1(): Entrenador + getJugador2(): Entrenador + getRondaActual(): int + resetearAnteriorEfectos(): void + calcularTipoEfecto(atacante: Digimon, defensor: Digimon): int + jugarRonda(d1: Digimon, d2: Digimon, habilidad1: boolean, habilidad2: boolean): int + determinarGanador(): int + resetearBatalla(): void

Propósito de la clase: Esta clase representa y administra una batalla entre los dos entrenadores. Es el núcleo de la lógica del combate: controla la ronda actual, calcula los efectos de tipo, aplica ataque, defensa, Digievoluciones y penalizaciones, conserva durante una ronda adicional los efectos activados, registra los ataques finales y determina los ganadores de rondas y de la batalla.

Propósito de los atributos:

- **digimonesDisponibles**: almacena el arreglo general de los ocho Digimon creados en Main y mantiene una referencia a los Digimon disponibles en el sistema.
- **jugador1**: almacena la referencia al primer Entrenador participante.
- **jugador2**: almacena la referencia al segundo Entrenador participante.
- **rondaActual**: indica el número de ronda que se está jugando. Inicia en uno y aumenta después de cada enfrentamiento.
- **ataqueFinaU1**: almacena el ataque total calculado para el Digimon del jugador 1 en la última ronda, permitiendo mostrarlo posteriormente desde Main.
- **ataqueFinaU2**: almacena el ataque total calculado para el Digimon del jugador 2 en la última ronda.
- **digievolucionActivadaJ1**: indica si el jugador 1 logró activar una nueva Digievolución en la ronda actual. Permite a Main mostrar el mensaje correspondiente.
- **digievolucionActivadaJ2**: cumple la misma función para el jugador 2.
- **efectoAnteriorDEvolucionJ1**: almacena el valor numérico de la Digievolución que el jugador 1 activó en la ronda anterior para aplicarlo una vez más en la ronda siguiente.

- efectoAnteriorDEvolucionJ2: almacena el valor numérico del efecto anterior del jugador 2.

- tipoEfectoAnteriorDEvolucionJ1: almacena qué tipo de efecto debe heredarse de la ronda anterior para el jugador 1: Ataque, Reduccion o Incrementar Defensa.

- tipoEfectoAnteriorDEvolucionJ2: almacena el tipo de efecto heredado de la ronda anterior para el jugador 2.

Propósito de los métodos:

- Constructor BatallaDigimon(digimonesDisponibles, jugador1, jugador2): recibe los Digimon generales y los dos entrenadores. Inicializa la ronda en uno, los ataques finales y efectos anteriores en cero, los indicadores de activación en falso y los tipos de efecto anterior como cadenas vacías.

- getAtaqueFinaU1(): retorna el ataque total calculado para el jugador 1 en la última ronda.

- getAtaqueFinaU2(): retorna el ataque total calculado para el jugador 2.

- getDigievolucionActivadaJ1(): retorna verdadero o falso según si la Digievolución del jugador 1 se activó en la ronda actual.

- getDigievolucionActivadaJ2(): retorna el estado de activación de la Digievolución del jugador 2.

- getDigimonesDisponibles(): retorna el arreglo general de Digimon.

- getJugador1(): retorna la referencia al primer entrenador.

- getJugador2(): retorna la referencia al segundo entrenador.

- getRondaActual(): retorna el número de la ronda actual para mostrarlo desde Main.

- resetearAnteriorEfectos(): coloca en cero los valores de efectos anteriores y vacía sus tipos para evitar que una Digievolución se aplique durante más rondas de las permitidas.

- calcularTipoEfecto(atacante, defensor): recibe dos Digimon y compara sus tipos. Retorna 20 cuando el atacante tiene ventaja, -10 cuando tiene desventaja y 0 cuando la relación es neutral.

- jugarRonda(d1, d2, habilidad1, habilidad2): recibe los dos Digimon seleccionados y dos booleanos que indican si cada entrenador desea intentar su Digievolución. Aplica efectos anteriores, intenta nuevas Digievoluciones, aplica penalización del 10% si fallan, calcula defensa y ataque total, incrementa la ronda ganada correspondiente y retorna 1 si gana jugador 1, 2 si gana jugador 2 o 0 si empatan.

- determinarGanador(): compara las rondas ganadas de ambos entrenadores después de la batalla. Incrementa la victoria del entrenador vencedor y retorna 1 para jugador 1, 2 para jugador 2 o 0 si existe empate.

- resetearBatalla(): restablece rondas ganadas, estado usado de los Digimon, efectos anteriores, indicadores de Digievolución, ataques finales y rondaActual para permitir otra batalla con los mismos equipos.

Main
+ main(args: String[]): void

Propósito de la clase: Esta clase contiene el punto de entrada del programa y se encarga de toda la interacción con los usuarios. Crea las Digievoluciones y los ocho Digimon, permite formar los dos equipos, crea los entrenadores, muestra el menú principal, solicita las decisiones de cada ronda, ejecuta la batalla y presenta todos los mensajes y resultados en consola.

Propósito de los atributos: ningún atributo

Propósito de los métodos:

- main(args): recibe el arreglo de argumentos estándar de Java. Dentro del método se crea el Scanner, las Digievoluciones, los Digimon y los equipos; se solicitan los nombres de los entrenadores; se valida la selección de los cuatro Digimon de cada jugador; se muestra el menú principal; se ejecutan las cuatro rondas de una batalla; se pregunta por el uso de Digievolución; se muestran los ataques y ganadores; se reinicia la batalla cuando corresponde y se permite terminar el programa.

